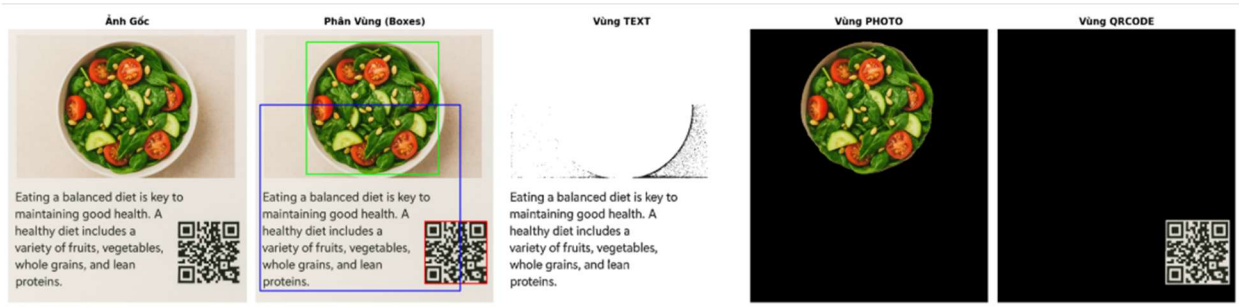
```python
def process_final_assignment(img_path):
    # 1. Setup
    img = cv2.imread(img_path)
    if img is None:
        print(f"Lỗi: Không tìm thấy ảnh tại {img_path}")
        return

    # Copy ảnh để vẽ Bounding Boxes (Step 6)
    output_bbox = img.copy()
    h, w = img.shape[:2]

    # --- STEP 1: Input --> Gray Scale --> Binary (Otsu threshold)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    # Blur nhẹ để giảm nhiễu
    gray_blur = cv2.GaussianBlur(gray, (5, 5), 0)
    # Otsu Thresholding
    _, binary = cv2.threshold(gray_blur, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)

    # --- STEP 2: Detect QR Code --> qrcode_mask
    qrcode_mask = np.zeros_like(binary)
    detector = cv2.QRCodeDetector()
    retval, points, _ = detector.detectAndDecode(gray)

    qr_box = None
    if points is not None:
        points = points.astype(int)
        # Tạo mask QR
        cv2.fillPoly(qrcode_mask, [points], 255)
        # Dilate nhẹ để mask bao trùm hết viền QR
        qrcode_mask = cv2.dilate(qrcode_mask, np.ones((5,5), np.uint8))
        qr_box = cv2.boundingRect(points)

    # --- STEP 3: Remove QR region from binary image
    # binary_no_qr = binary - qrcode
    binary_no_qr = cv2.subtract(binary, qrcode_mask)
```

```python
# --- STEP 4: Split Photo region using Morphology
photo_mask = np.zeros_like(binary)
kernel_morph = cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3))

# Thực hiện phép Morphology (Erosion) để tách dính
eroded_img = cv2.morphologyEx(binary_no_qr, cv2.MORPH_ERODE, kernel_morph, iterations=4)

# Tìm contours trên ảnh đã tách
cnts_core, _ = cv2.findContours(eroded_img, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

if len(cnts_core) > 0:
    # Lấy contour lớn nhất (lõi của Salad)
    salad_core = max(cnts_core, key=cv2.contourArea)

    # Kỹ thuật Reconstruction: Tìm contour gốc từ ảnh binary chưa bị mòn
    cnts_orig, _ = cv2.findContours(binary_no_qr, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    for cnt in cnts_orig:
        if cv2.contourArea(cnt) < 500: continue

        # Kiểm tra va chạm: Nếu contour gốc chứa lõi salad -> Đó là Photo
        M = cv2.moments(salad_core)
        if M["m00"] != 0:
            cX, cY = int(M["m10"]/M["m00"]), int(M["m01"]/M["m00"])
            if cv2.pointPolygonTest(cnt, (cX, cY), False) >= 0:
                cv2.drawContours(photo_mask, [cnt], -1, 255, -1)
                # Vẽ Box cho Photo (Step 6 later)
                px, py, pw, ph = cv2.boundingRect(cnt)
                cv2.rectangle(output_bbox, (px, py), (px+pw, py+ph), (0, 255, 0), 2)
                break

# --- STEP 5: Split Text region using subtract
# 1. Trừ Photo ra khỏi Binary gốc
text_temp = cv2.subtract(binary, photo_mask)

# 2. Trừ tiếp QR ra khỏi kết quả trên
text_final_mask = cv2.subtract(text_temp, qrcode_mask)

# Lọc nhiễu nhẹ cho Text mask
text_final_mask = cv2.morphologyEx(text_final_mask, cv2.MORPH_OPEN, np.ones((2,2), np.uint8))
```

```python
# --- STEP 6: Draw Bounding Boxes
# Vẽ khung QR
if qr_box is not None:
    qx, qy, qw, qh = qr_box
    cv2.rectangle(output_bbox, (qx, qy), (qx+qw, qy+qh), (0, 0, 255), 2)

# Vẽ khung Text
# Gom nhóm các ký tự text để vẽ 1 khung lớn
kernel_text_group = cv2.getStructuringElement(cv2.MORPH_RECT, (4, 1))
text_dilated = cv2.dilate(text_final_mask, kernel_text_group, iterations=2)
cnts_text, _ = cv2.findContours(text_dilated, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

all_text_points = []
for c in cnts_text:
    if cv2.contourArea(c) > 20: all_text_points.append(c)

# Biến lưu ảnh kết quả hiển thị Text (xử lý màu)
img_text_display = np.ones_like(img) * 255

if len(all_text_points) > 0:
    all_points = np.vstack(all_text_points)
    tx, ty, tw, th = cv2.boundingRect(all_points)
    # Padding
    pad = 5
    tx, ty = max(0, tx - pad), max(0, ty - pad)
    tw, th = min(w - tx, tw + 2*pad), min(h - ty, th + 2*pad)

    # Vẽ Box Text (Màu Xanh Dương)
    cv2.rectangle(output_bbox, (tx, ty), (tx+tw, ty+th), (255, 0, 0), 2)

    # Visualization: Xử lý màu chữ (đen nhẹ)
    roi_color = img[ty:ty+th, tx:tx+tw]
    roi_gray = gray[ty:ty+th, tx:tx+tw]

    # Mask vùng cấm (để xóa rác viền)
    forbidden = cv2.bitwise_or(photo_mask, qrcode_mask)
    forbidden = cv2.dilate(forbidden, np.ones((17,17), np.uint8)) # Tẩy mạnh
    roi_forbidden = forbidden[ty:ty+th, tx:tx+tw]

    # Adaptive Threshold lấy chữ
    roi_bin = cv2.adaptiveThreshold(roi_gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, 21, 10)
    mask_text_roi = cv2.bitwise_not(roi_bin)
    mask_text_roi[roi_forbidden == 255] = 0 # Xóa rác

    # Tạo nền trắng
    bg_white = cv2.bitwise_and(np.ones_like(roi_color)*255, np.ones_like(roi_color)*255, mask=cv2.bitwise_not(mask_text_roi))

    # Tạo chữ màu đen nhẹ (20, 20, 20)
    solid_soft_black = np.full_like(roi_color, (20, 20, 20))
    fg_text = cv2.bitwise_and(solid_soft_black, solid_soft_black, mask=mask_text_roi)

    # Gộp lại
    final_roi = cv2.add(bg_white, fg_text)
    img_text_display[ty:ty+th, tx:tx+tw] = final_roi
```

```python
    # --- In các output
    img_salad_only = cv2.bitwise_and(img, img, mask=photo_mask)
    img_qr_only = cv2.bitwise_and(img, img, mask=cv2.dilate(qrcode_mask, np.ones((3,3), np.uint8)))

    fig, axes = plt.subplots(1, 5, figsize=(20, 6))
    titles = ["Ảnh Gốc", "Phân Vùng (Boxes)", "Vùng TEXT", "Vùng PHOTO", "Vùng QRCODE"]
    images = [img, output_bbox, img_text_display, img_salad_only, img_qr_only]

    for i in range(5):
        axes[i].imshow(cv2.cvtColor(images[i], cv2.COLOR_BGR2RGB))
        axes[i].set_title(titles[i], fontsize=11, fontweight='bold')
        axes[i].axis('off')
        for spine in axes[i].spines.values():
            spine.set_edgecolor('gray'); spine.set_linewidth(1)

    plt.tight_layout()
    plt.show()

# --- Gọi hàm
if __name__ == "__main__":
    process_final_assignment('input.png')
```